

TITLE PAGE

- **Problem Statement ID** – Question 4
- **Problem Statement Title-** AI-Powered Behavioral Anomaly Detection for Cybersecurity
- **Theme-** Cybersecurity
- **PS Category-** Software
- **Student Name (Registered on portal)-** ALOUKIK DAS
- **Student ID-** 23BCE10551 (College)

SOC Sentinel: Dual-Model Behavioral Anomaly Engine

Proposed Solution:-

- Detailed explanation of the proposed solution:-
 - A localized, real-time threat detection engine designed to analyze access-log telemetry without relying on static signatures.
 - Utilizes a two-stage machine learning ensemble: an unsupervised model establishes a behavioral baseline, and a supervised classifier categorizes specific threat vectors.
 - Translates mathematical model confidence directly into an actionable 0-100 Risk Score for Security Operations Center (SOC) analysts.
- How it addresses the problem:-
 - **Imbalance:** Bypasses the class imbalance problem by using outlier detection (Isolation Forest) to isolate anomalies before classification.
 - **UX/Explainability:** Replaces binary "alert/no-alert" outputs with context-rich explanations (e.g., AI confidence percentages, specific attack labels) inside a filterable React dashboard
- Innovation and uniqueness of the solution:-
 - **Hardware-Efficient:** Explicitly designed to run inference on edge-level hardware (tested on an Intel Core i3) without heavy recurrent networks.
 - **Cold-Start Resilient:** Evaluates events against global telemetry baselines rather than requiring historical data for every new user or device.

TECHNICAL APPROACH

- Technologies to be used:-

- **Core ML:** Python, Scikit-Learn (Isolation Forest, Random Forest Classifier), Pandas, NumPy.
- **Backend & Data:** FastAPI (async log streaming), SQLite, SQLAlchemy.
- **Frontend UI:** React.js, Tailwind CSS, Recharts (dynamic telemetry visualization), Lucide Icons.

- Methodology and process for implementation:-

- **Phase 1 (Data Ingestion):** Pandas-driven generator creates synthetic user/device profiles and injects exact threat taxonomies (Brute Force, Impossible Travel, Lateral Movement).
- **Phase 2 (Feature Engineering):** Raw logs converted into behavioral vectors (session duration, geo-velocity, failure rates).
- **Phase 3 (Inference Pipeline):**
 - **Step A:** Isolation Forest calculates statistical deviation from normal traffic.
 - **Step B:** If deviation threshold is met, Random Forest classifies the exact attack type.
- **Phase 4 (Actionable Output):** FastAPI pushes scored alerts to the React UI, rendering real-time radar animations and generating downloadable CSV threat reports.

FEASIBILITY AND VIABILITY

- Analysis of the feasibility of the idea:-

- **Highly Feasible & Scalable:** By utilizing Scikit-Learn ensemble methods instead of heavy deep learning (LSTMs), the engine requires minimal compute overhead. It successfully runs real-time inference on entry-level edge hardware (e.g., Intel Core i3).
- **Immediate Deployment:** The inclusion of a robust synthetic data generator allows security teams to test, validate, and tune the models immediately without waiting weeks for organic log collection.

- Potential challenges and risks:-

- **Concept Drift:** Legitimate enterprise behavior changes over time (e.g., employees shifting departments, new devices), which could trigger false positives.
- **The Cold-Start Problem:** Accurately scoring access events for brand-new employees or devices that possess no historical baseline.

- Strategies for overcoming these challenges:-

- **Addressing Drift:** The architecture supports sliding-window retraining. The Isolation Forest can be cheaply retrained on a nightly batch of the last 7-14 days of logs to naturally adapt to evolving behavioral baselines.
- **Addressing Cold-Starts:** The pipeline evaluates new events against the global enterprise baseline rather than strictly individual histories. Unseen categorical variables are handled safely via zero-indexed fallback encoders.

ARTIFACTS

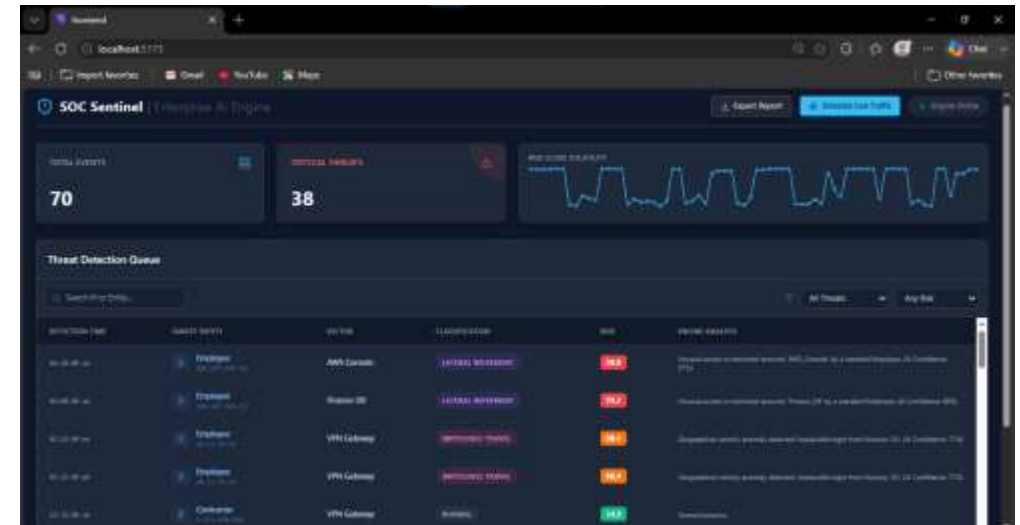
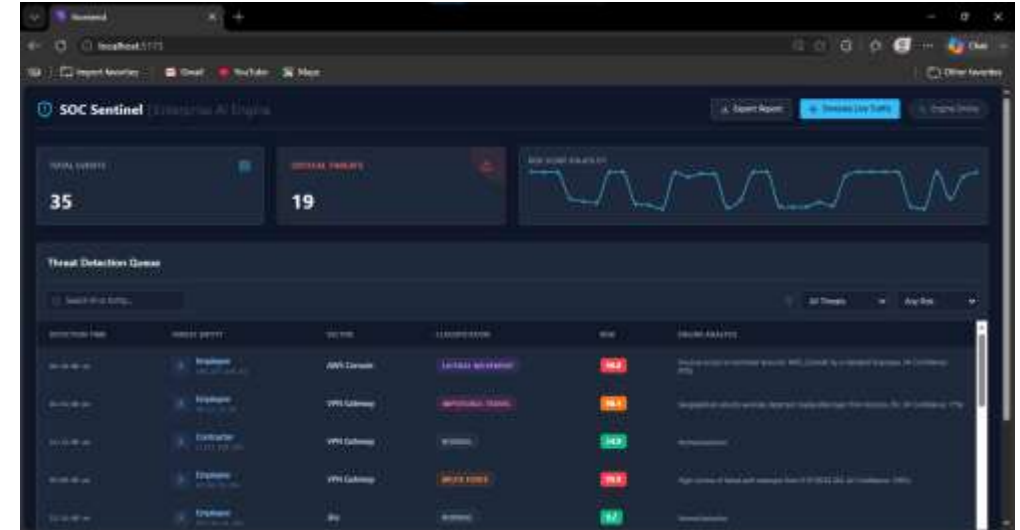
- detector.py(predict_proba code):-

```
class AnomalyDetector:
    def analyze_event(self, log_data: dict) -> dict:
        features = self._engineer_single_event(log_data)
        base_score = self.lis_forest.decision_function(features)[0]
        base_risk = floor(100 - ((base_score - (-0.3)) / (0.3 - (-0.3)) * 100))
        base_risk = max(0.0, min(100.0, base_risk))
        is_anomaly = bool(self.lis_forest.predict(features)[0] == -1)
        result = {
            'is_anomaly': is_anomaly,
            'risk_score': round(base_risk, 2),
            'anomaly_type': 'normal',
            'explanation': 'Normal behavior.'
        }
        probabilities = self.classifier.predict_proba(features)[0]
        classes = self.classifier.classes
        attack_probs = {cls: prob for cls, prob in zip(classes, probabilities) if cls != 'normal'}
        if attack_probs:
            top_attack = max(attack_probs, key=attack_probs.get)
            ai_confidence = float(attack_probs[top_attack])
            if ai_confidence > 0.40:
                result['is_anomaly'] = True
                result['anomaly_type'] = top_attack
                base_exp = self._generate_explanation(top_attack, log_data)
                result['explanation'] = f'{base_exp} (AI Confidence: {int(ai_confidence * 100)}%)'
                calculated_risk = 0.0 + (ai_confidence * 40)
                feature_jitter = (log_data.get('session duration', 0) * 100) / 100.0
                result['risk_score'] = round(min(99.9, calculated_risk + feature_jitter), 1)
        return result
```

- Simulation Snap:-



- Dashboard Snap:-



RESEARCH AND REFERENCES

- Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). *Isolation Forest*. IEEE International Conference on Data Mining. Applied for baseline outlier detection.
- Scikit-Learn Developers (2024). *Ensemble Methods: Random Forests*. Used for supervised threat categorization and dynamic risk score mapping via model confidence (predict_proba).
- MITRE ATT&CK Framework. Used as the theoretical basis for defining the synthetic attack taxonomies (Lateral Movement, Brute Force, Impossible Travel).
- NIST Special Publication 800-207. Principles applied to continuous behavioral monitoring of authenticated sessions.